# Homework 1 – Color Transform

314552046 莊明憲

## SPEC

- Please represent "lena.png" in terms of RGB, YUV, and YCbCr.

1. RGB -> YUV:

$$
\begin{aligned}
Y &= 0.299 * R + 0.587 * G + 0.114 * B \\
U &= -0.169 * R - 0.331 * G + 0.5 * B + 128 \\
V &= 0.5 * R - 0.419 * G - 0.081 * B + 128
\end{aligned}
$$

2. RGB -> YCbCr: in the slides

- In any programming language you are comfortable with (C/C++/Python/MATLAB).
- Output 8 grayscale images representing R, G, B, Y, U, V, Cb, and Cr, respectively.
- **Do not** use any ready-made functions to transform the color.
- You are allowed to use image reading/writing APIs.
- Deadline: 2025/09/29 13:19.
- Compressed as a single ZIP file.
- Required files :

1. **VC_HW1_[student_id].pdf**: Report PDF
2. **VC_HW1_[student_id].zip**: Source code (C/C++/Python/MATLAB) with a **README** file instructing the TAs on how to run your code.

## Code Explanation

### 1. Initial setup

Load the source image lena.png safely, convert it into floating point Numpy array, and ensures that the output directory for results exists.

```python
def rgb_to_yuv_ycbcr_processing(image_path='lena.png'):
    #  讀取並準備圖片
    try:
        img = Image.open(image_path)
        # 確保影像是 RGB 格式，以便分離通道
        img_rgb = img.convert('RGB')
        rgb_array = np.array(img_rgb, dtype=np.float64)
    except FileNotFoundError:
        print(f"錯誤：找不到圖片檔案 '{image_path}'。請確保它在正確的路徑下。")
        return

    # 建立資料夾存放輸出圖片
    output_dir = 'final_output_images'
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
```

## 2. Color space transformation

First, it separates R, G, B channels from the input image data. It then applies the two sets of math formulas provided in slides and spec to compute the Y, U, V, Cb, Cr color space channels.

```
# 分離 R, G, B 通道
R = rgb_array[:, :, 0]
G = rgb_array[:, :, 1]
B = rgb_array[:, :, 2]

# RGB -> YUV 轉換
Y = 0.299 * R + 0.587 * G + 0.114 * B
U = -0.169 * R - 0.331 * G + 0.5 * B + 128
V = 0.5 * R - 0.419 * G - 0.081 * B + 128

# RGB -> YCbCr 轉換
Cb = -0.148 * R - 0.291 * G + 0.439 * B + 128
Cr = 0.439 * R - 0.368 * G - 0.071 * B + 128
```

## 3. Image saving

It gathers all eight computed image channels (R, G, B, Y, U, V, Cb, Cr) into a dictionary. The code then iterates through this dictionary, converting each channel's data into a standard 8-bit grayscale image and saving it as a separate .png file.

```
# 儲存要求的 8 個通道
channels_to_save = {
    'R_channel': R,
    'G_channel': G,
    'B_channel': B,
    'Y_channel': Y,
    'U_channel': U,
    'V_channel': V,
    'Cb_channel': Cb,
    'Cr_channel': Cr,
}

# 儲存 8 張灰階圖
for name, data in channels_to_save.items():
    img_channel = Image.fromarray(data.astype(np.uint8), 'L')
    save_path = os.path.join(output_dir, f'{name}.png')
    img_channel.save(save_path)
    print(f"已儲存: {save_path}")
```

## 4. Main block

This is the main entry point of the Python script. When the program is run, this block of code calls the primary processing function to start the entire color conversion and image saving process.

```
if __name__ == '__main__':
    rgb_to_yuv_ycbcr_processing()
```

---

## Result

| | |
|---|---|
| | |
| R | G |
| | |
| B | Y |

| | |
|---|---|
| | |
| U | V |
| | |
| Cb | Cr |